ASOCIACIÓN ASTRONÓMICA
Y EDUCATIVA DE CANARIAS
Henrietta Swan Leavitt

# UNA VENTANA AL UNIVERSO

# 2025-2026

-

# E5: Introduction to orbits and black holes

-

## Worksheet 1: Plotting Newtonian Orbits with Python

David Sola Gil
(AAEC)

# 1. Introduction

Newton's Law of Universal Gravitation constitutes an essential pillar of theoretical physics, standing out for its elegant simplicity and its universal character. Despite the paradigm shift with the arrival of General Relativity about 200 years later, it continues to capture, with great qualitative and quantitative precision, the dynamics of celestial bodies in our Solar System.

This worksheet has been designed to highlight the value of physics as an analytical tool, taking Newton's formulation as its central axis. The main objective is to show how the integration of Python allows for the enrichment of the learning process: starting from abstract mathematical expressions, students will create their own visualizations, thus facilitating a deeper and more tangible understanding of gravitational phenomena.

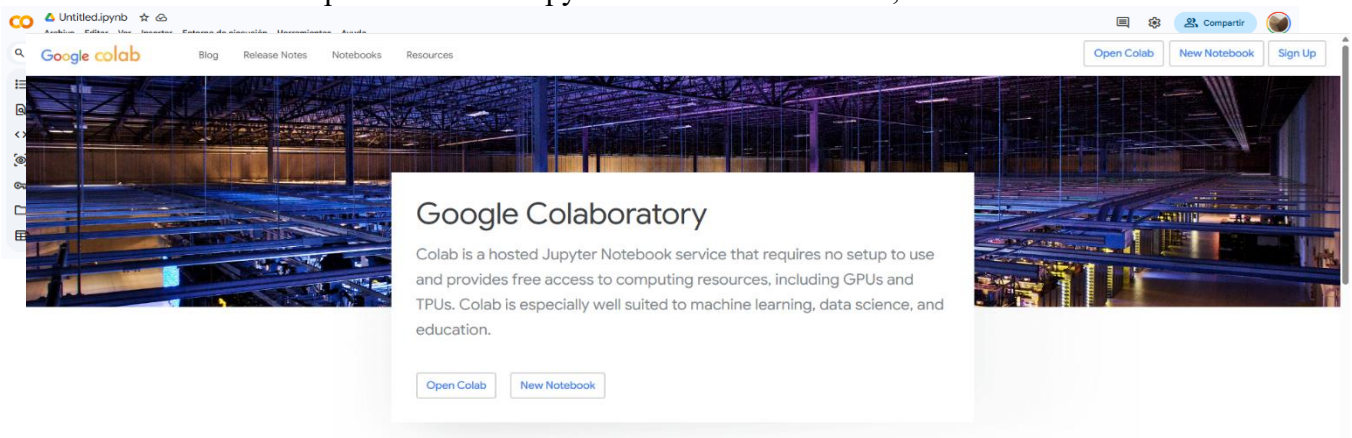
# 2. Warm up: conic sections

**Step 0: Creating a Jupyter Notebook and Initial Plots**

Before starting with graphical representations, it is advisable that you perform a short warm-up in Python to familiarize yourself with the coding tools, their possibilities, and their limitations. Your starting point will be Google Colab, a web platform that allows you to execute code without the need for prior installations.

- If you have a Google account with an active session, you can save the files to your online account.
- If not, you can download them locally for future use. Additionally, students in the Canary Islands can use it starting from age 14 with their @canariaseducacion.es account.

For all the advantages mentioned above, it is a perfect environment to bring programming into the classroom. The first step is to create a Jupyter Notebook. To do this, click on New Notebook.




From here, we will go to a web page where we will see an options bar and a blank cell. The blank cell is where we can write Python code.

Recall that Python uses libraries, which are code files with functions designed to facilitate calculations and professional graphical representations. Two of the most used, which you will load in your first step, are Numpy (for numerical calculations) and Matplotlib (for visualizations).

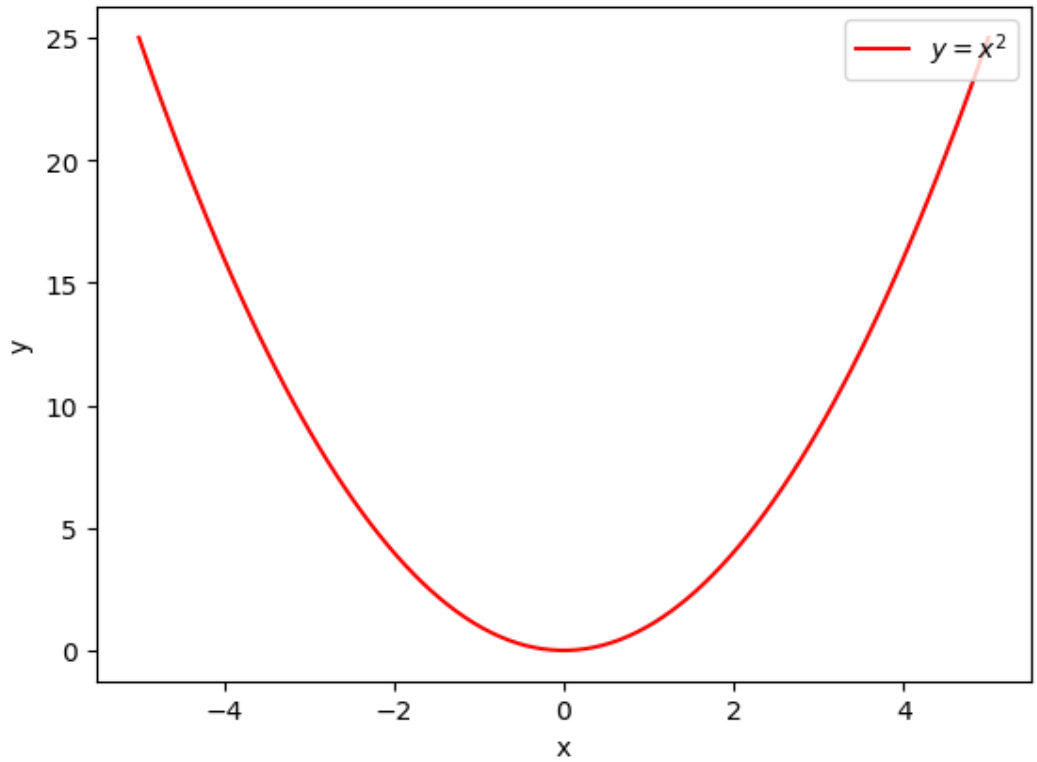
```
import numpy as np
import matplotlib.pyplot as plt
```

Here we are loading the *numpy* library and the *pyplot* module within matplotlib, giving them the aliases *np* and *plt* (standard aliases) so that every time we access those libraries, we do not have to write the full name.

Our first exercise is to use Python to draw a plot of a parabola. This is done with the following code:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)
y = x**2
plt.plot(x,y, color='red')
plt.legend(['$y=x^2$'], loc="upper right")
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```




Here we have

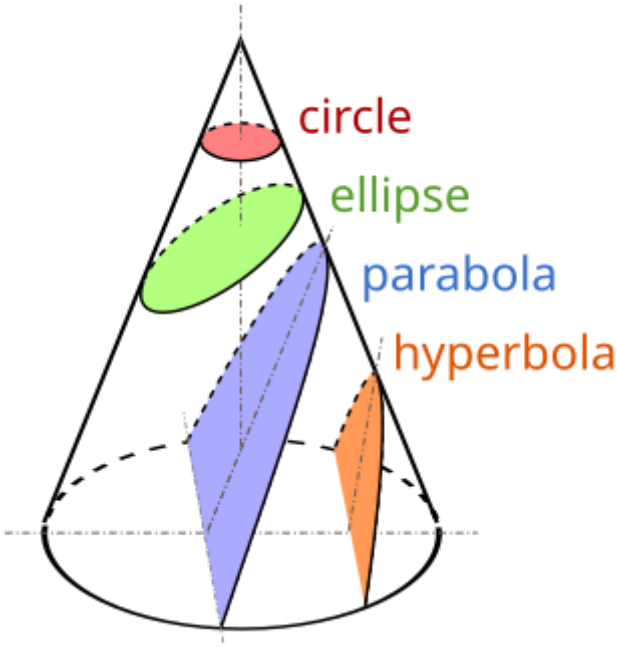
1. Loaded the libraries.
2. Used the function np.linspace(-5, 5, 100) to create a vector of points x, ranging from x = -5 tp x = 5 in 100 steps.
3. Squared each of those points to obtain a list of values y.
4. Created the plot y = x^2 by joining these points.
5. Added a legend to the plot.
6. Added labels to both axes.
7. Displayed the plot.

As a first exercise, copy the code above into Python to draw a parabola. Once generated, I suggest you modify the code to represent the function y = x^3 in the domain (-10, 10). Experiment by reducing the number of points or changing the colour to blue; these tests will help you familiarize yourself with how the tool functions.

For a more detailed description of everything that can be done with Matplotlib, consult the official documentation.

**Step 1: Drawing Conic Sections**

As seen in the theoretical content of the module, the orbits of celestial bodies under Newton's gravity are described by conic sections: circle, ellipse, parabola, and hyperbola, shown in the graphic below. To complete this introductory block, I propose that you represent these curves in the $(x, y)$ plane.




Our starting point is the ordinary equations for each conic section:

1. Parabola: $y = 2px^2$, with $p$ constant.
2. Ellipse (and the circumference as a special case):

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1,$$

where $a$ y $b$ are constants, and the circumference corresponds to $a = b$.

3. Hyperbola:

$$\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1,$$

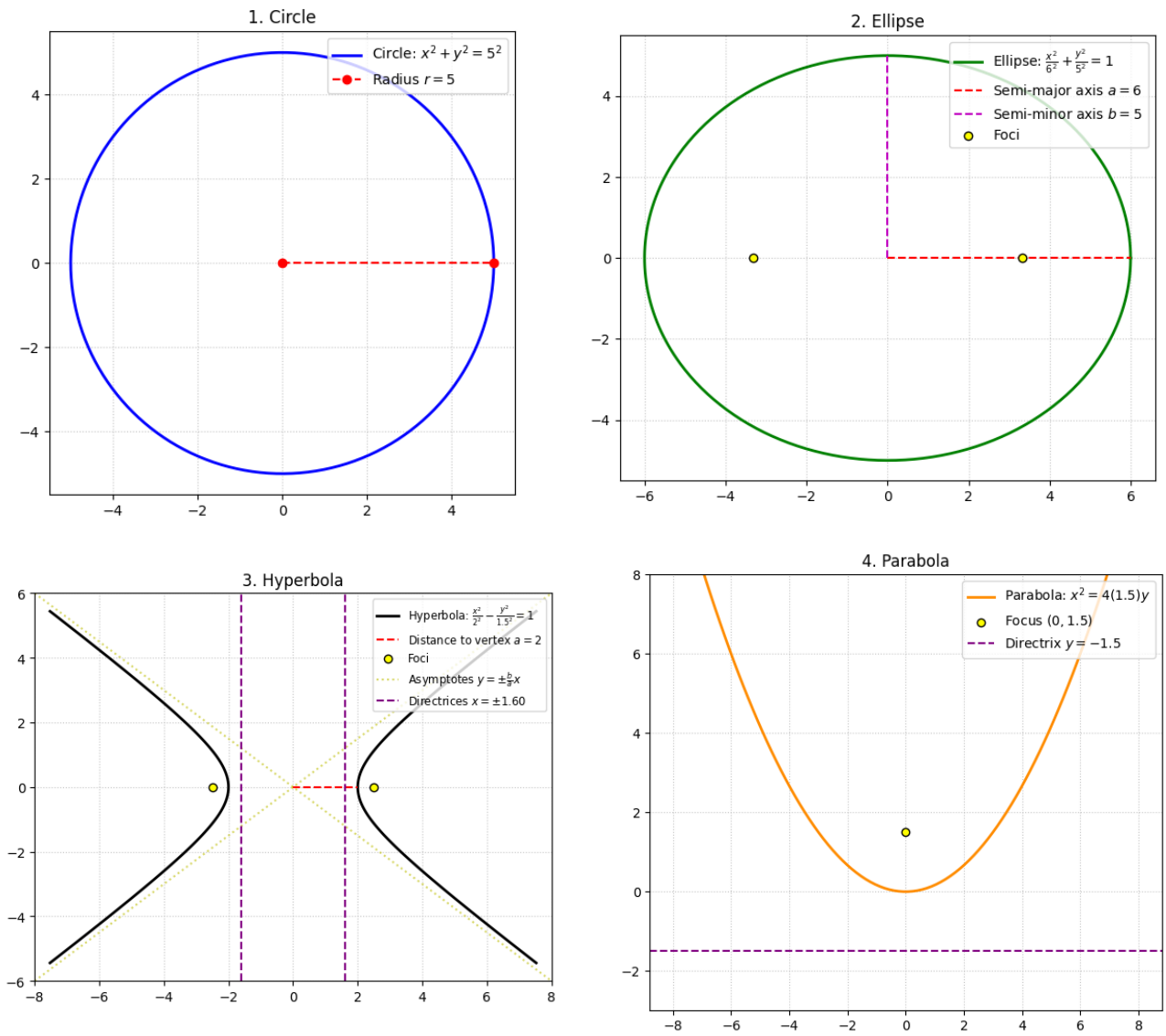
where $a$ y $b$ are constants.

Write a small piece of code in Python that draws an example of each of the conic sections. Although you can try to draw them through "brute force" by dividing the plane, you will find it much more elegant to

use their parametric form, expressing $(x, y)$ in terms of an angle $\phi \in [0, 2\pi)$. As a hint, remember the following trigonometric function equations:

$$\cos^2(\phi) + \sin^2(\phi) = 1\,,$$
$$\cosh^2(\phi) - \sinh^2(\phi) = 1.$$

The final result should be similar to the figure below.




How many constants do I need to draw each conic section? How does changing each of the constants affect each plot?

# 3. The Effective Potential

After familiarizing yourself with the conics, begin your journey toward the representation of orbits. A fundamental step before solving any equation is to attempt to extract the maximum information from

the problem through analysis.

With this objective, you will study the energy equation $E$ of a celestial body of mass $m$, located at a distance $r$ from the attracting body of mass $M$. As explained in the notes, $E$ is constant and is given by:

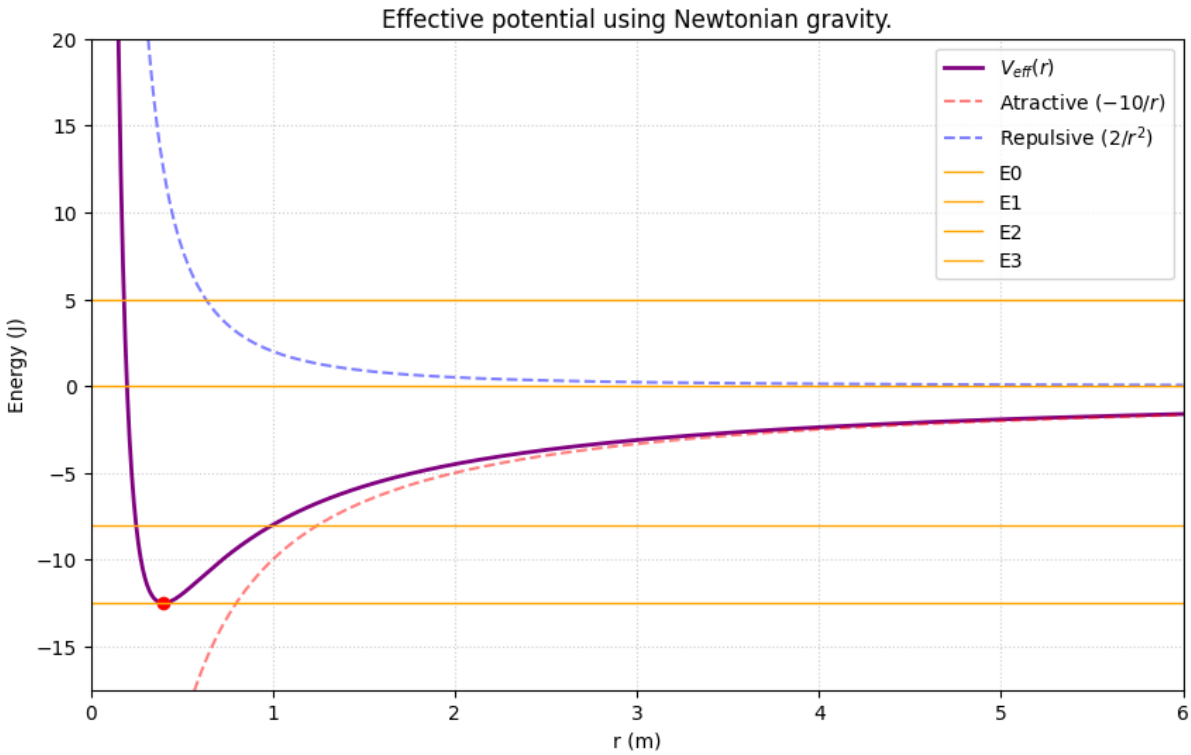
$$E = \frac{1}{2}m\left(\frac{dr}{dt}\right)^2 - \frac{mk}{r} + \frac{ml^2}{2r^2}.$$

Here $l$ is the angular momentum per unit mass, y $k = GM$, where $G = 6.674 \times 10^{-11}\ N \cdot \frac{m^2}{kg^2}$. For our purposes, all these variables are constants. We are going to graphically represent the effective potential (everything that is not kinetic energy), which is:

$$V_{eff}(r) = -\frac{mk}{r} + \frac{ml^2}{2r^2}.$$

Our goal now is to perform a study of this function. Draw, in Python, a plot of this function. Before writing code, answer the following questions:

1. What happens to the function as $r \to 0$? And for $r \to \infty$?
2. Are there local extrema? Where and of what type?
3. What happens if $l = 0$? What is the physical interpretation of this?




To get a nice-looking plot, I have used the constants $m = 1, k = 20, l = 2$.

By studying the plot, we are going to deduce what type of orbit we have according to the energy $E$ of the system. We will denote the minimum energy via $E_{min}$. Choose the conic section that describes the orbit in each case.

1. $E = E_{min}$.
2. $E_{min} < E < 0$.
3. $E > 0$.
4. $E = 0$.

*Hint: To do this exercise, it is convenient to imagine horizontal lines representing the constant energy E superimposed on the previous plot. Where E intersects $V_{eff}$, we can interpret that the kinetic energy is 0. What happens if I then increase the kinetic energy a little?*

# 4. Drawing orbits

Once the parameters are understood, you can proceed to the representation of the orbits. Unlike the initial exercises, you will observe that these conic sections are not centred at the origin and are expressed more efficiently in polar coordinates:

$$r(\phi) = \frac{r_0}{e\cos(\phi) + 1},$$

where $r_0 = l^2/k$ is a constant and $e \in [0, \infty)$ is another constant known as *eccentricity*. These sections are also expressed using:

$$x = r\cos(\phi), \quad y = r\sin(\phi).$$

Draw the orbits for $r_0 = 1$ and eccentricities

1. $e = 0$.
2. $0 < e < 1$.
3. $e = 1$.
4. $e > 1$.

Now try increasing or reducing $r_0 \in (0, \infty)$. How do the plots vary? Where is the Sun located?

Remember that in section 2.2 of the notes, we connected eccentricity and conic sections with the energy value and the orbits they produce through:

$$E = \frac{mk^2}{2l^2}(e^2 - 1)$$

Use this equation as an alternative method to verify that the answers from the previous exercise match!

# 5. Comparing Orbits

As a final stage, you will compare the Newtonian estimate with real orbital data. Although estimating orbits is a highly complex process requiring advanced models, you will see that Newton's mechanics captures the movement with a remarkably low error for many cases.

For these exercises, we will use NASA data from an online database known as JPL Horizons, which contains information on planets, asteroids, satellites, and comets. It is free. Shocking, isn't it?

The first step is to install the library in your Google Colab session to access this information. Simply modify the beginning of your code so that it installs the corresponding library.

```
!pip install -q astroquery
import numpy as np
import matplotlib.pyplot as plt
from astroquery.jplhorizons import Horizons
```

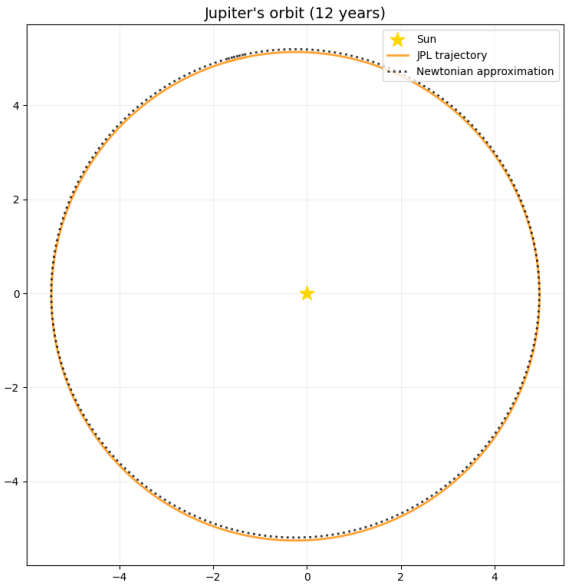
You have examples of how to load planetary positions and velocities using jplhorizons in the corresponding Python file. These data come in a dictionary format. In the Moodle Jupyter Notebook, there are functions written that read the data for the desired celestial body to extract the parameters for the orbit equation and save you the work of conversion.

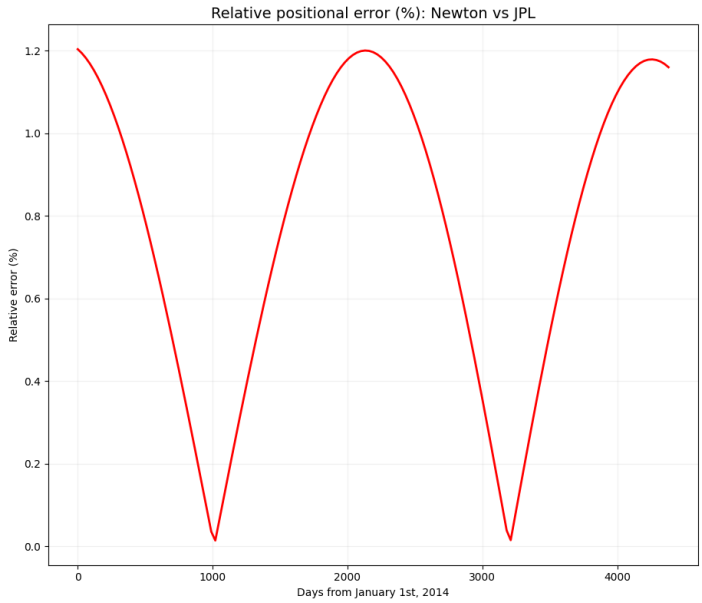
After installing the necessary library in your Google Colab session, I propose two challenges:

1. Using the initial position and velocity of, for example, Jupiter, estimate its orbit for 12 years. Compare the result of the Newtonian approximation with the real data.

2. In 2019, amateur astronomer Gennady Borisov observed the first comet from outside our solar system: 2I/Borisov. Use JPL data to estimate its trajectory through the solar system using a hyperbola.

From here, sky is the limit (figuratively). Use your creativity to expand this exercise in the classroom.




# Bibliography

Conic sections: https://en.wikipedia.org/wiki/Conic_section#/media/File:Conic_Sections.svg